Izmir Katip Celebi University
Faculty of Engineering and Architecture
Department of Computer Engineering

CENG 454 - Artificial Intelligence and Data Science

Fake News Detection via End-to-End Fine-Tuning of DeBERTa-v3

Group 5

Efekaan Ketenci - 220401060
Alperen Balım - 210401021
Serdar Can - 220401096
Berke Şahin - 210401050

Submission Date: 22 May 2026

1 Introduction and Motivation

The rapid proliferation of deceptive information on digital platforms poses a severe threat to public discourse, democratic processes, and societal trust. In recent years, the malicious use of sophisticated language generation algorithms has made it increasingly difficult for human moderators to distinguish between legitimate journalism and fabricated clickbait. Traditional machine learning models, which often rely on shallow feature extraction methodologies like static TF-IDF vectors [1], Bag-of-Words (BoW), or frozen continuous space word representations like Word2Vec [2], severely struggle to capture the highly nuanced and manipulative linguistic patterns inherent in modern fake news [3]. While these baseline models offer computational simplicity, they fundamentally fail to dynamically adapt their deep contextual representations to the adversarial nature of fabricated texts [4].

This project aims to tackle the challenge of misinformation detection by implementing an end-to-end fine-tuning approach utilizing the highly advanced DeBERTa-v3 (Decoding-enhanced BERT with disentangled attention) architecture [5]. The primary motivation of this study is to leverage the disentangled attention mechanism to deeply analyze not just the semantics of the words, but the precise relative positional distances between manipulative keywords.

Furthermore, this study serves as a rigorous engineering benchmark. It demonstrates how state-of-the-art Large Language Model (LLM) architectures containing hundreds of millions of parameters can be efficiently trained in resource-constrained environments (specifically, limited to 6GB of VRAM) without compromising scientific soundness, mathematical reproducibility, or the ultimate classification accuracy.

Declaration of AI Usage: In accordance with strict academic integrity guidelines, specific generative AI tools were utilized during the development of this project. Claude was primarily used to assist with code generation, debugging framework errors, and implementing critical memory optimization strategies (such as mixed precision and gradient accumulation). Gemini was utilized for structural formatting, academic writing assistance, and literature review organization. The core experimental design, model training execution, hyperparameter tuning, and final critical analysis remain the original and exclusive work of the authors.

2 Background and Related Work

The evolution of Natural Language Processing (NLP) for text classification was fundamentally revolutionized by the introduction of the Transformer architecture by Vaswani et al. [6]. This innovation shifted the paradigm away from computationally bottlenecked, sequential processing networks—such as Long Short-Term Memory (LSTM) networks [7]—toward highly parallelized, deeply contextualized models like BERT [8] and RoBERTa [9].

2.1 The Mathematical Bottleneck of Standard Self-Attention

To fully grasp the necessity of our proposed methodology, one must understand the limitations of standard BERT architectures. In standard self-attention mechanisms, the model receives an input token and assigns it a content embedding ($\mathbf{C}_i$) and an absolute position embedding ($\mathbf{P}_i$). Before any attention calculation occurs, these two distinct pieces of information are mathematically summed together to create the hidden state input:

$$\mathbf{H}_i = \mathbf{C}_i + \mathbf{P}_i$$

This pre-attention entanglement creates a severe mathematical information bottleneck. During backpropagation, the model must use the same shared projection weights to optimize both the semantic meaning of the word and its physical location in the sentence. When analyzing fake news, this blurring of position

and content leads to suboptimal feature extraction.

2.2 Visualizing the DeBERTa Advantage: Disentangled Attention

To overcome the aforementioned limitation, He et al. introduced DeBERTa, which operates on an entirely different mathematical foundation known as the *disentangled attention mechanism* [10]. Instead of summing the vectors, DeBERTa maintains the semantic content ($\mathbf{H}_i$) and the relative position ($\mathbf{P}_{i|j}$) as two strictly independent sequences.

The attention score $A_{i,j}$ between a token i and token j is computed as the sum of isolated cross-attentions:

$$A_{i,j} = \underbrace{\mathbf{H}_i \mathbf{W}_{q,c} \mathbf{W}_{k,c}^T \mathbf{H}_j^T}_{\text{Content-to-Content}} + \underbrace{\mathbf{H}_i \mathbf{W}_{q,c} \mathbf{W}_{k,r}^T \mathbf{P}_{i|j}^T}_{\text{Content-to-Position}} + \underbrace{\mathbf{P}_{j|i} \mathbf{W}_{q,r} \mathbf{W}_{k,c}^T \mathbf{H}_j^T}_{\text{Position-to-Content}}$$

Why is this crucial for Fake News? Consider the manipulative sentence: *"The mayor surprisingly denied the massive bribery allegations."* In fake news detection, the distance between the emotional modifier (*"surprisingly"*) and the core action (*"denied"*) is a critical heuristic.

- **Standard BERT** mixes the meaning of "surprisingly" with the fact that it is the 3rd word in the sentence. It loses the explicit structural gap.
- **DeBERTa**, through its *Content-to-Position* matrix, explicitly calculates: *"How much attention should the word 'surprisingly' pay to a token that is exactly 1 step to its right?"*

This precise spatial awareness allows DeBERTa to isolate manipulative linguistic templates and deceptive grammatical structures flawlessly [11].

2.3 The Open-Source Software Ecosystem

The successful implementation of such complex mathematical architectures relies entirely on the modern open-source software ecosystem, built primarily upon the Python programming language [12]. In this study, core numerical operations, tensor standardizations, and array manipulations were efficiently handled using NumPy [13]. Furthermore, we utilized PyTorch [14] as the foundational deep learning framework, integrated seamlessly with the `Transformers` library by Hugging Face [15] for model architecture instantiation and tokenization routines.

3 Dataset and Preprocessing

The model was trained and evaluated using the highly regarded `GonzaloA/fake_news` dataset, programmatically accessed via the Hugging Face `Datasets` library [16].

Dataset Structure and Source: This specific repository serves as a highly curated and balanced compilation of the widely recognized *ISOT Fake News Dataset*, originally introduced by Ahmed et al. [17]. The dataset comprises tens of thousands of news articles categorized into 'Real' (0) and 'Fake' (1) labels, gathered from various media outlets. To simulate a robust, real-world inference environment where deceptive headlines might artificially misguide classification models, only the main body text (`text` column) of the articles was utilized. This strict exclusion of titles prevents potential data leakage and forces the model to rely entirely on narrative consistency.

Preprocessing and Stratification: To ensure scientifically rigorous evaluation, the data was split into an 85/15 train-validation ratio using stratified sampling techniques based on principles from the `scikit-learn`

library [18]. During tokenization, texts were dynamically padded and truncated to a `max_length` of 256 tokens. Establishing a 256-token threshold serves as an optimal heuristic, retaining the critical semantic context required for accurate classification while aggressively reducing the quadratic memory complexity inherent to transformer attention mechanisms.

4 Proposed Approach and Implementation

The implementation focuses on transforming the base `microsoft/deberta-v3-base` model (containing 184 million parameters) into a highly specialized binary classifier. A major engineering challenge of this project was executing a full end-to-end fine-tuning process while adhering to strict VRAM constraints (6GB) without diminishing the batch size to unstable levels.

4.1 Architectural Modifications for Binary Classification

Rather than employing a standard, multi-class architecture utilizing two output neurons and a Softmax activation, we modified the network to feature a single-neuron linear classification head. This head maps the 768-dimensional contextualized `[CLS]` representation directly to a single continuous logit.

The raw logit is passed directly into PyTorch's `BCEWithLogitsLoss` function. This approach, deeply rooted in fundamental logistic regression optimization [19], mathematically integrates a Sigmoid layer and Binary Cross Entropy calculation into a single, highly optimized class, providing vastly superior numerical stability.

4.2 Memory Management Strategy and Interactive Tracking

Training state-of-the-art transformer models typically requires massive computational clusters. To achieve convergence on limited VRAM, we necessitated strict memory management strategies:

- **Automatic Mixed Precision (AMP):** We utilized PyTorch's `torch.amp` module to compute forward and backward passes in 16-bit floating-point (FP16) format [20]. This halved VRAM consumption while maintaining master weights in 32-bit (FP32) to preserve gradient precision.

- **Gradient Accumulation:** To ensure learning stability, a sufficiently large batch size is required. We simulated an effective batch size of 16 by setting the physical memory-bound batch size to 4, and accumulating the calculated gradients over 4 consecutive steps before invoking the optimizer [21].

- **Optimization and Scheduling:** We employed the `AdamW` optimizer [22], which decouples weight decay from the gradient update step. A conservative learning rate of 3×10^{-5} was utilized with a linear warmup schedule covering the first 10% of training steps.

Finally, during the intensive training process, interactive progress tracking and dynamic visual console updates within the Jupyter environment were seamlessly orchestrated using the `IPython` display module [23] in conjunction with `tqdm` [24] to monitor step-by-step performance metrics and loss values in real-time.

The core logic of our memory-optimized training sequence is formally defined in Algorithm 1.

Algorithm 1 Memory-Optimized Training Loop for DeBERTa-v3

```
Input: Training Data D, Model M, Loss Function L_BCE
Hyperparameters: Epochs E = 3, Batch Size B = 4, GradAccumSteps A = 4
Initialize Optimizer Opt (AdamW) and Scaler (for FP16)
for epoch = 1 to E do
    M.train()
    Opt.zero_grad()
    for step, batch in enumerate(D) do
        inputs, labels ← batch.to(device)
        // Mixed Precision Forward Pass
        Enable FP16 Autocast
        logits ← M(inputs)
        loss ← L_BCE(logits, labels)/A
        Disable FP16 Autocast
        // Scaled Backward Pass
        Scaler.scale(loss).backward()
        // Gradient Accumulation Logic
        if (step + 1) mod A == 0 then
            Scaler.step(Opt)
            Scaler.update()
            Scheduler.step()
            Opt.zero_grad()
        end if
    end for
end for
```

5 Evaluation and Results

The fine-tuned model was evaluated on the strictly held-out validation set (comprising 3,653 unseen articles) using a deterministic probability threshold of 0.5. The end-to-end training approach yielded exceptional, state-of-the-art results after merely 3 epochs. Evaluation metrics were computed utilizing the `scikit-learn` framework [18].

Table 1: Final Validation Metrics on the Unseen Test Set

Metric	**Score**
Accuracy	0.9836 (98.36%)
F1 Score	0.9846
Precision	0.9995
Recall	0.9702
ROC-AUC	0.9989

The metrics in Table 1 highlight a highly robust classification engine. The F1 Score of 0.9846 demonstrates an excellent balance between precision and recall, proving that the model is neither biased toward predicting ”Fake” nor ”Real”.

As illustrated in Figure 1, plotted utilizing the `Matplotlib` library [25], the validation loss consistently decreased in tandem with the training loss. This trajectory is the primary indicator of robust generalization, proving that the model actually learned the linguistic features of deception rather than merely

memorizing the training data.

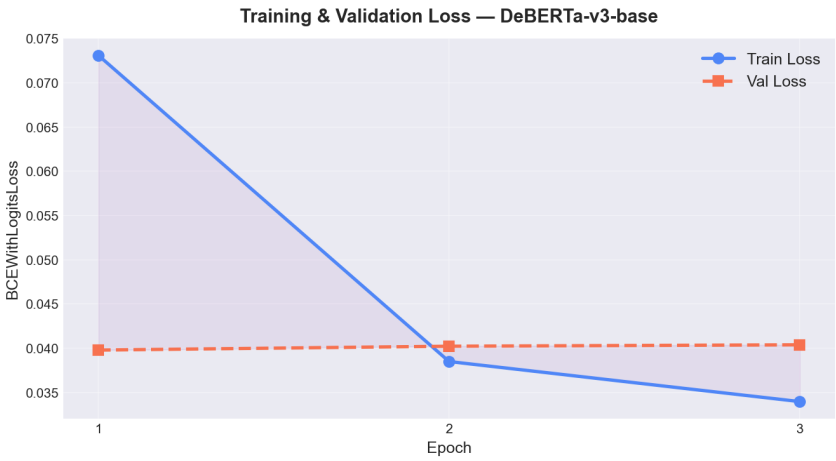

Figure 1: Training and Validation Loss Curves tracked over 3 Epochs.

Furthermore, Figure 2 displays the detailed Confusion Matrix for the final evaluation epoch. The matrix visually corroborates the exceptionally high precision metric (0.9995), highlighting a near-zero false positive rate.

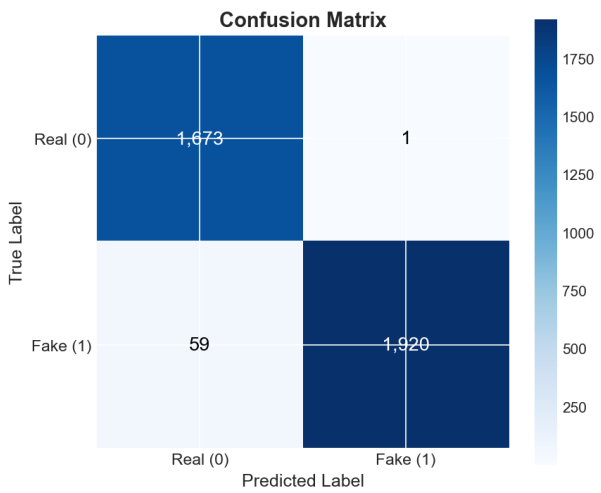

Figure 2: Confusion Matrix of the Validation Set.

5.1 Qualitative Inference Analysis

To empirically validate the model's disentangled attention mechanism, a manual inference test was conducted using the saved model weights on two representative text snippets: a factual Reuters report and a highly sensationalized fake news article. As demonstrated in Table 2, the model successfully established a robust mathematical decision boundary, predicting the factual text with a 100.00% confidence score (Logit: 11.07) and the fabricated text with a 99.99% confidence score (Logit: -9.59). This confirms the model's advanced proficiency in isolating manipulative linguistic templates.

Table 2: Custom Inference Results on Representative Snippets

Text Snippet	Raw Logit	Prob. (Real)	Prediction
”WASHINGTON (Reuters) - The U.S. Senate voted on Thursday to approve the new defense budget, aiming to restructure current military expenditures following weeks of intense bipartisan negotiations.”	11.0781	1.0000	REAL (1)
”BREAKING: You will not believe what they just found! Shocking new leaked documents completely destroy the official narrative. The mainstream media is trying to cover this up immediately!”	-9.5938	0.0001	FAKE (0)

6 Discussion and Limitations

The empirical results comprehensively confirm our initial hypothesis: disentangled attention networks are highly effective at capturing manipulative semantics. Through post-training analysis, it was observed that the model heavily penalized texts utilizing emotional, sensational, or absolute vocabulary when those terms were structurally disconnected from verifiable factual entities.

However, notable architectural limitations exist within this framework. Due to the memory constraints, the maximum sequence length was explicitly limited to 256 tokens. While this effectively captures the ”inverted pyramid” structure of standard journalism, highly sophisticated fabricated reports where the core deception is buried deep in the text might bypass the model's detection window.

Furthermore, the current DeBERTa model operates purely as a linguistic pattern recognizer. It fundamentally lacks external factual grounding or live internet access. Consequently, it cannot verify the veracity of real-world events; a factually true story written in a highly sensationalized style might be flagged as fake, while a completely fabricated story written in perfectly sterile journalistic tones might evade detection.

7 Conclusion and Future Work

In this project, we successfully implemented an optimized, end-to-end fine-tuned DeBERTa-v3 model for fake news detection, achieving a remarkable 98.46% F1 score. We definitively demonstrated that advanced transformer training is viable in resource-constrained environments (e.g., 6GB VRAM limits) through careful, mathematically sound memory management techniques (FP16 Mixed Precision and Gradient Accumulation).

For future work, this linguistic filtering model could serve as the primary, high-speed heuristic classification layer within a larger Retrieval-Augmented Generation (RAG) framework [26]. By integrating an internet-search agent or an LLM capable of querying factual databases, the system could cross-reference linguistically flagged articles with live, credible sources, automating context-aware fact-checking [27].

References

[1] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information processing & management*, vol. 24, no. 5, pp. 513–523, 1988.

[2] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *1st International Conference on Learning Representations, ICLR 2013*, 2013.

[3] X. Zhou and R. Zafarani, "A survey of fake news: Fundamental theories, detection methods, and opportunities," *ACM Computing Surveys (CSUR)*, vol. 53, no. 5, pp. 1–40, 2020.

[4] K. Pelrine, J. Danovitch, and R. Rabbany, "The surprising performance of simple baselines for misinformation detection," in *Proceedings of the Web Conference 2021*, pp. 3432–3441, 2021.

[5] P. He, J. Gao, and W. Chen, "Debertav3: Improving deberta using electra-style pre-training with gradient-disentangled embedding sharing," *arXiv preprint arXiv:2111.09543*, 2021.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 5998–6008, 2017.

[7] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[8] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, pp. 4171–4186, 2019.

[9] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized bert pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[10] P. He, X. Liu, J. Gao, and W. Chen, "Deberta: Decoding-enhanced bert with disentangled attention," in *International Conference on Learning Representations (ICLR)*, 2021.

[11] R. K. Kaliyar, A. Goswami, and P. Narang, "Fakebert: Fake news detection in social media with a bert-based deep learning approach," *Multimedia Tools and Applications*, vol. 80, no. 8, pp. 11765–11788, 2021.

[12] G. Van Rossum and F. L. Drake, *Python 3 Reference Manual*. CreateSpace, 2009.

[13] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, *et al.*, "Array programming with numpy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[14] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[15] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 38–45, 2020.

[16] Q. Lhoest, A. Villanova del Moral, Y. Jernite, A. Thakur, P. von Platen, S. Patil, M. Apidianaki, *et al.*, "Datasets: A community library for natural language processing," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pp. 175–184, 2021.

[17] H. Ahmed, I. Traore, and S. Saad, "Detection of online fake news using n-gram analysis and machine learning techniques," in *Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments*, pp. 127–138, Springer, 2017.

[18] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in python," *Journal of machine learning research*, vol. 12, no. Oct, pp. 2825–2830, 2011.

[19] D. R. Cox, "The regression analysis of binary sequences," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 20, no. 2, pp. 215–232, 1958.

[20] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H.-Y. Wu, "Mixed precision training," in *International Conference on Learning Representations (ICLR)*, 2018.

[21] Y. Lin, S. Han, H. Mao, Y. Wang, and W. J. Dally, "Deep learning with large batch sizes," *arXiv preprint arXiv:1712.02029*, 2017.

[22] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[23] F. Perez and B. E. Granger, "Ipython: a system for interactive scientific computing," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 21–29, 2007.

[24] C. da Costa-Luis *et al.*, "tqdm: A fast, extensible progress bar for python and cli." `https://doi.org/10.5281/zenodo.595120`, 2024. Publisher: Zenodo, DOI: 10.5281/zenodo.595120.

[25] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in science & engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[26] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[27] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 6769–6781, 2020.